TRAVAUX PRATIQUES

Détection de contours : méthodes classiques et deep learning

Vision par ordinateur

Problématique. Les opérateurs classiques détectent efficacement des variations locales d'intensité, mais peuvent produire des contours fragmentés ou sensibles au bruit. Un réseau profond tel que HED apprend des représentations multi-échelles. Peut-on combiner les deux approches pour obtenir une carte de contours plus exploitable ?

Document étudiant — énoncé, code de départ, expériences et questions d'analyse

— title : "Travaux pratiques — Détection de contours" subtitle : "Combiner les méthodes classiques et le deep learning" author : "Vision par ordinateur" lang : fr-FR —

Objectifs pédagogiques

À la fin du TP, vous devrez être capables de :

- mettre en œuvre les détecteurs de contours Sobel et Canny ; - utiliser un modèle profond pré-entraîné HED (*Holistically-Nested Edge Detection*) avec OpenCV DNN ; - normaliser et fusionner deux cartes de contours ; - comparer les résultats avec des critères visuels et des mesures simples ; - discuter les avantages, les limites et le coût des approches classiques, profondes et hybrides.

Niveau : Licence avancée / Master **Durée :** 1 h 30 à 2 h **Organisation :** travail en binôme conseillé.

1. Problématique

Les opérateurs classiques détectent efficacement les variations locales d'intensité, mais peuvent produire des contours fragmentés ou être sensibles au bruit. Un réseau profond tel que HED apprend des représentations multi-échelles à partir d'images. Peut-on combiner les deux approches pour obtenir une carte de contours plus exploitable ?

2. Rappels

Sobel approxime le gradient de l'image. Une forte norme du gradient indique une variation locale importante, susceptible de correspondre à un contour. Canny enchaîne généralement un lissage, le calcul du gradient, la suppression des non-maxima et un seuillage par hystérésis.

HED exploite plusieurs niveaux de représentation d'un réseau convolutif afin de prédire une carte de contours. Dans ce TP, le modèle est pré-entraîné : il n'est pas nécessaire de l'entraîner.

Soient $C(x, y)$ la carte Canny normalisée et $H(x, y)$ la carte HED normalisée, toutes deux dans $[0, 1]$. On construit la fusion :

$$F(x, y) = \alpha H(x, y) + (1 - \alpha)C(x, y)$$

Le paramètre α règle l'importance de la prédiction profonde. Une valeur de 0,6 donne davantage de poids à HED, sans supprimer l'information de Canny. Cette fusion est une expérience de départ : elle n'est pas garantie d'être meilleure sur toutes les images.

3. Préparation de l'environnement

Installez les bibliothèques nécessaires dans un terminal ou une cellule Jupyter :

```
python -m pip install opencv-python numpy matplotlib
```

Choisissez une image contenant plusieurs types de contours : objets, textures, lignes ou zones ombrées. Exemples : une rue, des feuilles, des objets sur une table ou une scène architecturale. Utilisez si possible une image libre de droits ou une photo prise par vous-même. Placez-la dans le même dossier que le script et nommez-la ‘image.jpg‘.

Fichiers du modèle HED

Le modèle HED pour OpenCV DNN nécessite un fichier de définition de réseau (‘prototxt‘) et des poids pré-entraînés (‘caffemodel‘). Téléchargez ces deux fichiers depuis le dépôt officiel ‘opencv$_extra$‘, $dansledossierd'exemplesHED, puisplacez-lesdansledossierduTPsouslesnomssuivants$:

- ‘deploy.prototxt‘ - ‘hed$_pretrained_bsds.caffemodel$‘

Référence : [dépôt OpenCV Extra](https ://github.com/opencv/opencv$_extra/tree/master/testdata/cv/ximgp$

Arborescence attendue :

```
TP_contours/
  image.jpg
  tp_contours.py
  deploy.prototxt
  hed_pretrained_bsds.caffemodel
```

4. Travail demandé

Partie A — Prétraitement et méthode classique (20 min)

1. Charger l'image en niveaux de gris et en couleur. 2. Appliquer un flou gaussien léger afin de limiter le bruit. 3. Calculer les contours avec Sobel puis Canny. 4. Tester au moins deux couples

de seuils Canny, par exemple $(50, 150)$ et $(100, 200)$. 5. Afficher l'image originale, Sobel et les deux résultats Canny.

Partie B — Détection par deep learning (25 min)

1. Charger le réseau HED avec OpenCV DNN. 2. Exécuter une prédiction sur l'image. 3. Ramener la sortie à la taille de l'image et normaliser les valeurs entre 0 et 1. 4. Afficher la carte HED en niveaux de gris. 5. Décrire deux différences observées par rapport à Canny : continuité, détails, textures, contours faibles ou faux positifs.

Partie C — Fusion hybride (20 min)

1. Normaliser la carte Canny et la carte HED. 2. Calculer $F = \alpha H + (1 - \alpha)C$ pour $\alpha = 0{,}3$, $0{,}6$ et $0{,}8$. 3. Afficher les trois fusions. 4. Choisir le résultat le plus utile pour l'image étudiée et justifier le choix.

Partie D — Analyse et mini-compte rendu (25 min)

Construire un tableau comparatif avec les colonnes suivantes : méthode, continuité des contours, sensibilité aux textures/bruit, coût d'exécution et observations. Inclure les images produites et répondre aux questions de la section 6.

5. Code Python de départ

Le code ci-dessous constitue une base complète. Complétez les chemins des fichiers si nécessaire. La partie HED dépend de la disponibilité des deux fichiers du modèle.

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

IMAGE_PATH = "image.jpg"
PROTO_PATH = "deploy.prototxt"
MODEL_PATH = "hed_pretrained_bsds.caffemodel"

# 1) Chargement et prtraitement
img = cv2.imread(IMAGE_PATH)
if img is None:
    raise FileNotFoundError(f"Image introuvable : {IMAGE_PATH}")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 1.2)

# 2) Mthodes classiques : Sobel et Canny
gx = cv2.Sobel(blur, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(blur, cv2.CV_32F, 0, 1, ksize=3)
sobel = cv2.magnitude(gx, gy)
sobel = cv2.normalize(
    sobel, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)
canny = cv2.Canny(blur, 100, 200)

# 3) Deep learning : HED pr-entran
net = cv2.dnn.readNetFromCaffe(PROTO_PATH, MODEL_PATH)
h, w = img.shape[:2]
blob = cv2.dnn.blobFromImage(
    img, scalefactor=1.0, size=(w, h),
    mean=(104.00698793, 116.66876762, 122.67891434),
    swapRB=False, crop=False
)
net.setInput(blob)
hed_raw = net.forward()
hed = hed_raw[0, 0]
hed = cv2.resize(hed, (w, h))
hed = cv2.normalize(hed, None, 0, 1, cv2.NORM_MINMAX)

# 4) Fusion : normaliser Canny puis combiner
canny_norm = canny.astype(np.float32) / 255.0
alpha_values = [0.3, 0.6, 0.8]
```

```
fusions = [
    a * hed + (1 - a) * canny_norm
    for a in alpha_values
]

# 5) Visualisation
titles = [
    "Image originale", "Sobel", "Canny", "HED",
    "Fusion alpha=0.3", "Fusion alpha=0.6",
    "Fusion alpha=0.8"
]
images = [
    cv2.cvtColor(img, cv2.COLOR_BGR2RGB),
    sobel, canny, hed
] + fusions

plt.figure(figsize=(14, 9))
for i, (title, im) in enumerate(zip(titles, images), start=1):
    plt.subplot(2, 4, i)
    plt.imshow(im, cmap=None if i == 1 else "gray")
    plt.title(title)
    plt.axis("off")
plt.tight_layout()
plt.show()

# Sauvegarder les rsultats pour le compte rendu
cv2.imwrite("resultat_sobel.png", sobel)
cv2.imwrite("resultat_canny.png", canny)
cv2.imwrite(
    "resultat_hed.png",
    (hed * 255).clip(0, 255).astype(np.uint8)
)
for a, f in zip(alpha_values, fusions):
    cv2.imwrite(
        f"fusion_{a:.1f}.png",
        (f * 255).clip(0, 255).astype(np.uint8)
    )
```

6. Questions d'analyse

1. Pourquoi applique-t-on un flou gaussien avant Canny ? Que se passe-t-il si le flou est trop important ? 2. Quel est l'effet d'une augmentation des seuils de Canny sur le nombre de contours détectés ? 3. HED produit-il uniquement des contours très nets de type binaire ? Comment interpréter les niveaux de gris de sa sortie ? 4. Dans quelles zones HED apporte-t-il une information que Canny ne détecte pas, ou inversement ? 5. La fusion améliore-t-elle toujours le résultat ? Donnez un exemple où elle pourrait ajouter des contours indésirables. 6. Quel est l'effet du paramètre α ? Comparez $\alpha = 0{,}3$ et $\alpha = 0{,}8$ sur votre image. 7. Quelle méthode choisiriez-vous pour une application embarquée avec peu de ressources ? Justifiez. 8. Comment évaluer quantitativement les résultats si une carte de contours de référence (*vérité terrain*) est disponible ? Citez au moins deux métriques possibles.

7. Livrables

À remettre par binôme :

- le script Python complété ; - l'image originale et les cartes Sobel, Canny, HED et fusionnées ; - un tableau comparatif synthétique ; - des réponses argumentées aux questions 1 à 8 (environ 1 à 2 pages).

8. Extension facultative

Testez une autre image, idéalement avec un fond texturé. Comparez les temps d'exécution de Canny et HED sur la même machine (répéter plusieurs fois et calculer une moyenne). Vous pouvez aussi remplacer la fusion pondérée par une fusion logique : appliquer un seuil à HED, puis prendre l'union des pixels de contour avec Canny. Expliquez pourquoi cette variante peut produire davantage de contours.